



Modern Lower Bound Techniques in Database Theory and Constraint Satisfaction

Dániel Marx

marx@cispa.de

CISPA Helmholtz Center for Information Security
Saarbrücken, Germany

ABSTRACT

Conditional lower bounds based on $P \neq NP$, the Exponential-Time Hypothesis (ETH), or similar complexity assumptions can provide very useful information about what type of algorithms are likely to be possible. Ideally, such lower bounds would be able to demonstrate that the best known algorithms are essentially optimal and cannot be improved further. In this tutorial, we overview different types of lower bounds, and see how they can be applied to problems in database theory and constraint satisfaction.

CCS CONCEPTS

• **Theory of computation** → **Problems, reductions and completeness; Database theory.**

KEYWORDS

computational complexity; conditional lower bounds; exponential-time hypothesis; parameterized complexity; database queries; constraint satisfaction problems

ACM Reference Format:

Dániel Marx. 2021. Modern Lower Bound Techniques in Database Theory and Constraint Satisfaction. In *Proceedings of the 40th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS '21)*, June 20–25, 2021, Virtual Event, China. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3452021.3458814>

1 INTRODUCTION

The design of efficient algorithms is in the focus of a large part of theoretical computer science research. The practical need to solve computational problems efficiently makes the systematic study of algorithmic efficiency highly motivated. Decades of research in algorithm design discovered mathematically beautiful and sometimes very practical algorithmic techniques that gave us deep insights into efficient computation in a wide range of contexts and application domains. The field of computational complexity treats algorithmic problems and computation as formal mathematical objects and tries to prove relationships between them [8, 55].

Given the abundance of computation in our modern world, it is justified to consider algorithms and computation as a fundamental

mathematical objects, on par with basic objects in geometry, algebra, and combinatorics. Researchers in computational complexity try to learn as much as possible about the mathematical nature of computation. But more pragmatically, computational complexity can give very important messages to algorithm designers. By giving information about limits of computation, it can prevent researchers from wasting time in dead ends of study: trying to design algorithm for problems that cannot be efficiently solved.

Having techniques to prove negative results can profoundly change the way research in algorithms is done. For example, the theory of NP-hardness changed the search for polynomial-time algorithms from a hit and miss effort to a more systematically doable project. Without NP-hardness, we would not be able to distinguish problems that do not admit polynomial-time algorithms from problems where we just were not yet successful in finding algorithms. But with the possibility of giving negative evidence in the form of NP-hardness, the lack of a known answer means that the question is still an active research problem: we typically expect that the algorithmic problem at hand can be eventually classified as either polynomial-time solvable or NP-hard, and it is worth trying to resolve the question one way or the other.

In a sense, computation complexity has not progressed much in the past 50 years despite intense efforts: the core questions underlying the hardness of computation, such as the celebrated $P \neq NP$ problem, are still wide open. However, by accepting certain well-chosen complexity assumptions, such as the $P \neq NP$ hypothesis, we can obtain *conditional lower bounds* explaining the apparent complexity of a large number of problems. As a general theme in computational complexity research, we can see a proliferation of new assumptions. These assumptions typically postulate that a certain type of algorithm does not exist for a particular fundamental problem (e.g., for Boolean satisfiability). The assumptions are chosen to be both plausible and have strong explanatory power: in many cases, they are able to show that the algorithms that we currently have are optimal and cannot be improved any further. Indeed, from the viewpoint of algorithm design, this is precisely the role of computation complexity: to separate problems where our current knowledge is complete from problems where there are still algorithmic ideas waiting to be discovered.

Some of the complexity assumptions are standard (such as $P \neq NP$), while others may be more controversial (such as the Strong Exponential-Time Hypothesis (SETH)). Therefore, the reader may wonder about the usefulness of proving conditional lower bounds based on unproven assumptions. It is important to point out that these conditional lower bounds are valuable even if we have doubts about the validity of the assumptions. Suppose we use a complexity assumption X to prove that a certain type of algorithm does not

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

PODS '21, June 20–25, 2021, Virtual Event, China

© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-8381-3/21/06...\$15.00

<https://doi.org/10.1145/3452021.3458814>

exists for a specialized problem P , perhaps in an application domain such as database theory. Even if we do not believe in the validity of the assumption X , this conditional lower bound shows that the difficulties we face when attacking problem P have nothing to do with the specific details of problem X or the application area: we are really facing assumption X in disguise and we need to disprove that first before any progress can be made on problem P . In other words, the conditional lower bound shows that we can stop trying to obtain the desired algorithm for problem P , as any such effort would be better spent on trying to disprove the (typically more fundamental) assumption X .

The purpose of this article is to highlight some of the lower bound techniques and show what kind of results they can deliver in the context of database theory. We will introduce a number of assumptions, contrast them, and show examples of their use. It has to be emphasized that this article does not aim to be an up to date survey of lower bounds in the area of database theory. The focus is more on the diverse set of assumptions and lower bound techniques that exist, rather than on presenting an exhaustive list of applications for each technique. Some of these example applications come directly from the literature on database query evaluation, but others were stated in essentially equivalent forms in other domains: for constraint satisfaction problems (CSP) or for graph-theoretic problems. Therefore, we begin with introducing the terminology for all these domains and then discuss the results using the most appropriate terminology.

2 THE FOUR DOMAINS

Conjunctive query evaluation is a fundamental problem in database theory. This problem can be equivalently seen as a CSP instance and therefore some of the results in the CSP literature are directly relevant. A large part of the CSP literature uses a formulation using the homomorphism of relational structures, which in some special cases degenerate to traditional graph problems. In this section, we introduce the terminology for all these domains and show how they are connected to each other.

2.1 Database queries

A join query Q is an expression of the form

$$R_1(a_{11}, \dots, a_{1r_1}) \bowtie \dots \bowtie R_m(a_{m1}, \dots, a_{mr_m}),$$

where the R_i are relation names with attributes $a_{i1}, \dots, a_{ir_i}$. Let A be the set of all attributes occurring in Q and $n = |A|$. A database instance $\mathbf{D}$ for Q consists of a domain $\text{dom}(\mathbf{D})$ and relations $R_i(\mathbf{D}) \subseteq \text{dom}(\mathbf{D})^{r_i}$ of arity r_i . It is common to think of the relation $R_i(\mathbf{D})$ as a table whose columns are labeled by the attributes $a_{i1}, \dots, a_{ir_i}$ and whose rows are the tuples in the relation. The answer, or set of solutions, of the query Q in $\mathbf{D}$ is the n -ary relation $Q(\mathbf{D})$ with attributes A consisting of all tuples t whose projection on the attributes of R_i belongs to the relation $R_i(\mathbf{D})$, for all i . Given the query Q and the database $\mathbf{D}$, the task in the JOIN QUERY problem is to compute the set $Q(\mathbf{D})$. In the BOOLEAN JOIN QUERY problem, we only need to decide if $Q(\mathbf{D})$ is empty or not. One can also define the counting version of the problem (i.e., compute $|Q(\mathbf{D})|$).

The primal graph of the query has the set A of attributes as vertex set and two variables are adjacent if there is a relation containing

both of them. The hypergraph of the instance is defined similarly: the vertex set is A , and each relation $R_i(a_{i1}, \dots, a_{ir_i})$ is represented by a hyperedge $\{a_{i1}, \dots, a_{ir_i}\}$.

2.2 Constraint satisfaction problems

Constraint satisfaction is a general framework that includes many standard algorithmic problems such as satisfiability, graph coloring, database queries, etc. A constraint satisfaction problem (CSP) instance consists of a set V of variables, a domain D , and a set C of constraints, where each constraint is a relation on a subset of the variables. The task is to assign a value from D to each variable in such a way that every constraint is satisfied. For example, 3SAT can be interpreted as a CSP instance where the domain is $\{0, 1\}$ and the constraints in C correspond to the clauses (thus the arity of each constraint is 3).

Formally, an instance I of a constraint satisfaction problem is a triple $I = (V, D, C)$, where:

- V is a set of variables,
- D is a domain of values,
- C is a set of constraints, $\{c_1, c_2, \dots, c_q\}$. Each constraint $c_i \in C$ is a pair $\langle s_i, R_i \rangle$, where:
 - s_i is a tuple of variables of length m_i , called the *constraint scope*, and
 - R_i is an m_i -ary relation over D , called the *constraint relation*.

For each constraint $\langle s_i, R_i \rangle$ the tuples of R_i indicate the allowed combinations of simultaneous values for the variables in s_i . The length m_i of the tuple s_i is called the *arity* of the constraint. A solution to a constraint satisfaction problem instance is a function f from the set of variables V to the domain D of values such that for each constraint $\langle s_i, R_i \rangle$ with $s_i = (v_{i1}, v_{i2}, \dots, v_{im_i})$, the tuple $(f(v_{i1}), f(v_{i2}), \dots, f(v_{im_i}))$ is a member of R_i . Given a CSP instance I , we can consider the problem of deciding if a solution exists, the problem of finding all solutions, or the problem of counting the number of solutions.

We say that an instance is *binary* if each constraint relation is binary, that is, $m_i = 2$ for every constraint¹. The *primal graph* (or *Gaifman graph*) of a CSP instance $I = (V, D, C)$ is a graph G with vertex set V , where $x, y \in V$ form an edge if and only if there is a constraint $\langle s_i, R_i \rangle \in C$ with $x, y \in s_i$. The hypergraph of an instance $I = (V, D, C)$ has V as its vertex set and for every constraint in C a hyperedge that consists of all variables occurring in the constraint.

Given a join query Q and a database $\mathbf{D}$, we can turn the query problem into a CSP instance I in a straightforward way: the domain of I is $\text{dom}(\mathbf{D})$, the set of variables correspond to the attributes A of Q , and for each relation R_i , there is a corresponding constraint c_i on the variables $a_{i1}, \dots, a_{ir_i}$. It is clear that the tuples in the answer set of Q in $\mathbf{D}$ are in one to one correspondence with the solutions of the CSP instance I . This establishes a correspondence between the basic algorithmic problems of the two domains.

It is worth pointing out that even though the two problems are equivalent, a large part of CSP research focuses on problem instances where the domain has small constant size and the number

¹It is unfortunate that while some communities use the term “binary CSP” in the sense that each constraint is binary (as does this dissertation), others use it in the sense that the variables are 0-1, that is, the domain size is 2.

of constraints is large (for example, 3SAT is such a problem). This has to be contrasted with the typical setting in database theory research where we assume that there are only a small number of attributes and relations have low arity, but the domain can be large and the number of tuples in a relation can be large.

2.3 Graph problems

Given a binary CSP instance $I = (V, D, C)$, we can equivalently formulate it as a graph problem. We construct a graph G the following way: let us introduce $|V| \cdot |D|$ vertices $w_{v,d}$ ($v \in V, d \in D$) and for every constraint $c_i = \langle (u, v), R_i \rangle$, let us make w_{u,d_1} and w_{v,d_2} adjacent if and only if $(d_1, d_2) \in R_i$. Let $W_i = \{w_{i,d} \mid d \in D\}$ and consider the partition $\mathcal{P} = \{W_1, \dots, W_{|V|}\}$. We say that a subgraph H of G *respects the partition $\mathcal{P}$* if every class of the partition $\mathcal{P}$ contains exactly one vertex of H .

Let $f : V \rightarrow D$ be a solution of I . If we consider the vertices $\{w_{v,f(v)} \mid v \in V\}$, then it is easy to see that they induce a subgraph H that respects $\mathcal{P}$ and is isomorphic to the primal graph of I . Conversely, it is not difficult to see that if G has a subgraph that respects $\mathcal{P}$ and is isomorphic to the primal graph of I , then it describes a solution of I . Therefore, the CSP instance can be described by an instance of *partitioned subgraph isomorphism*: given graphs H and G , and partition $\mathcal{P}$ of $V(G)$ into $|V(H)|$ classes, find a subgraph of V that respects $\mathcal{P}$ and is isomorphic to H . This problem is a natural variant of the standard *subgraph isomorphism* problem (find a subgraph of G isomorphic to H) and, as we have seen, its complexity is tightly connected to the complexity of CSP instances where the primal graph is H .

There is another way in which graph-theoretic notions can describe the solutions of a CSP instance. Consider a binary CSP instance $I = (V, D, C)$ where every constraint $c_i = \langle (u, v), R_i \rangle$ contains the same binary relation $R_i = R$, which we further assume to be symmetric (that is, $(d_1, d_2) \in R$ if and only if $(d_2, d_1) \in R$). Let H be the primal graph of I and let G be a graph with vertex set D where $d_1, d_2 \in D$ are adjacent if and only if $(d_1, d_2) \in R$. A *homomorphism from H to G* is a mapping $f : V(H) \rightarrow V(G)$ such that if u and v are adjacent in H , then $f(u)$ and $f(v)$ are adjacent in G . Note that f does not have to be injective (i.e., $f(u_1) = f(u_2)$ is possible) and if u and v are not adjacent, then we do not require that $f(u)$ and $f(v)$ be nonadjacent as well. It is easy to see that every solution $f : V \rightarrow D$ of I describes a homomorphism from H to G , in fact, these homomorphisms are in one to one correspondence with the solutions of I . Therefore, the complexity of finding a homomorphism from a fixed graph H to the input graph G is tightly connected to the complexity of CSP for instances with primal graph H where the same symmetric relation R appears in every constraint. If the relation R is not symmetric, then a similar connection can be made to the homomorphism problem in directed graphs.

2.4 Relational structures

The connection between CSP and graph homomorphisms that we have seen in the previous section has two major limitations: it worked only for binary CSP instances and only if every constraint contained the same relation R . These limitations can be removed if we move from graphs to the much more general setting of relational structures.

A *vocabulary* τ is a finite set of relation symbols of specified arities. The *arity* of τ is the maximum of the arities of all relational symbols it contains. A τ -structure $\mathbf{A}$ consists of a finite set A called the *universe* of $\mathbf{A}$ and for each relation symbol $R \in \tau$, say, of arity k , a k -ary relation $R^{\mathbf{A}} \subseteq A^k$. A *homomorphism* from a τ -structure $\mathbf{A}$ to a τ -structure $\mathbf{B}$ is a mapping $h : A \rightarrow B$ from the universe of $\mathbf{A}$ to the universe of $\mathbf{B}$ that preserves all relations, that is, for all $R \in \tau$, say, of arity k , and all tuples $(a_1, \dots, a_k) \in R^{\mathbf{A}}$ it holds that $(h(a_1), \dots, h(a_k)) \in R^{\mathbf{B}}$. Note that if τ contains only a single relational symbol, which has arity 2, then τ -structures are essentially directed graphs and the homomorphism problem between τ -structures is equivalent to the homomorphism problem on directed graphs.

More generally, we can express every CSP instance $I = (V, D, C)$ as a homomorphism problem the following way. Let $c = |C|$ be the number of constraints. Let the vocabulary τ contain c symbols $Q_1, \dots, Q_c$, where symbol Q_i has the same arity m_i as the constraint $c_i = \langle s_i, R_i \rangle$. We define τ -structure $\mathbf{A}$ over the universe V such that $Q_i^{\mathbf{A}}$ contains only the tuple s_i . We define τ -structure $\mathbf{B}$ over the universe D such that $Q_i^{\mathbf{B}}$ is precisely the relation R_i appearing in constraint c_i . Now it can be verified that a mapping $f : V \rightarrow D$ is a solution of I if and only if f is a homomorphism from $\mathbf{A}$ to $\mathbf{B}$.

3 UNCONDITIONAL LOWER BOUNDS

Ideally, we would like to prove negative results and lower bounds that are incontestably true mathematical statements. However, our inability to prove the $P \neq NP$ hypothesis is a major barrier that prevents us from proving most negative statements of interest. For all we know, it is still possible that $P = NP$ and we can solve all database query and CSP instances in polynomial time, and hence at the moment we cannot expect to unconditionally prove any result that rules out such algorithms. As long as we are in the classical setting of computation typically studied in computational complexity (algorithm is given an input, needs to compute a yes-no output), there is little hope in proving strong unconditional lower bounds.

We can hope to obtain unconditional lower bounds only if we deviate from the classical setting: for example, the problem involves the cost of accessing the input or the cost of communication. We show a particular, very simple setting in which we have tight unconditional lower bounds. If the task is to compute the answer to a join query, then the size of the answer is obviously an unconditional lower bound on the number of steps needed for computing the answer. This raises the question: what bounds can we give on the size of the answer and are there query evaluation algorithms that match this lower bound?

Formally, let $Q = R_1(a_{11}, \dots, a_{1r_1}) \bowtie \dots \bowtie R_m(a_{m1}, \dots, a_{mr_m})$ be a join query and let $\mathbf{D}$ be a database instance for Q such that every relation $R_i(\mathbf{D})$ contains at most N tuples. What can we say about the size of the answer? It is easy to see that N^m is an obvious upper bound: every tuple appearing in the answer chooses one of the at most N possibilities in each of the m relations. But this bound is often very far from being tight. For example, for the query $Q = R_1(a_1, a_2) \bowtie R_2(a_1, a_3) \bowtie R_3(a_2, a_3)$, it is known that the upper bound is $N^{3/2}$ instead of N^3 . The fractional number $3/2$ in the exponent of N suggests that obtaining the bound cannot be

completely obvious. Still, precise bounds can be obtained in a clean way using known combinatorial techniques.

Let us define the hypergraph H of the query Q the following way: the vertices are the attributes and we introduce a hyperedge $\{a_{i1}, \dots, a_{ir_i}\}$ for each relation $R_i(a_{i1}, \dots, a_{ir_i})$. A *fractional cover* of a hypergraph H is a mapping $f : V(H) \rightarrow [0, 1]$ such that for every vertex $v \in V(H)$, we have $\sum_{e \in E(H), v \in e} f(e) \geq 1$. That is, f is a weight assignment on the edges such that the total weight put on each vertex v is at least 1. The *weight* of f is $\sum_{e \in E(H)} f(e)$ and the *fractional edge cover number* $\rho^*(H)$ of H is the minimum weight of a fractional edge cover of H . For example, for the query $Q = R_1(a_1, a_2) \bowtie R_2(a_1, a_3) \bowtie R_3(a_2, a_3)$, the hypergraph H is a triangle and $\rho^*(H) = 3/2$ (assigning weight $1/2$ to each edge is a fractional edge cover and a quick analysis shows that this is optimal).

Using a simple application of Shearer's Lemma [24], which is a purely combinatorial statement on entropy, one can show that $N^{\rho^*(H)}$ is an upper bound on the number of solutions.

THEOREM 3.1 (ATSERIAS, GROHE, MARX [9]). *Let Q be a join query with hypergraph H . Let $\mathbf{D}$ be a database for Q where every relation has at most N tuples. Then the answer of Q in $\mathbf{D}$ has size at most $N^{\rho^*(H)}$.*

Conversely, we can show that $N^{\rho^*(H)}$ is essentially a tight lower bound. As it is usual with lower bound statements, we have to be a bit more careful with the formulation.

THEOREM 3.2 (ATSERIAS, GROHE, MARX [9]). *Let Q be a join query with hypergraph H . For infinitely many $N \geq 1$, there is a database $\mathbf{D}_N$ for Q where every relation has at most N tuples and the answer of Q in $\mathbf{D}_N$ has size at least $N^{\rho^*(H)}$.*

Theorem 3.2 provides an unconditional lower bound for any algorithm computing the full answer of Q (but of course it does not provide any bound on algorithms that just decide whether the answer is empty or compute the size of the answer). Are there algorithms that match this lower bound? The combinatorial proof of Theorem 3.1 can be turned into an algorithm with a constant overhead in the exponent, that is, to obtain $N^{\rho^*(H)+O(1)}$ running time. With additional techniques, it is possible to give tight algorithms that tightly match the lower bound.

THEOREM 3.3 ([54, 61]). *Let Q be a join query with hypergraph H . Let $\mathbf{D}$ be a database for Q where every relation has at most N tuples. Then the answer of Q in $\mathbf{D}$ can be computed in time $O(N^{\rho^*(H)})$.*

4 NP-HARDNESS

Since its conception and development in the early 70s [26, 38, 47], NP-hardness has been the main workhorse of providing intractability results for computational problems. The class NP contains decision problems that can be solved in nondeterministic polynomial time. This robust definition covers (the decision version of) most combinatorial and optimization problems of interest. If a problem P is NP-hard, then this means in particular that a polynomial-time algorithm for P would give polynomial-time algorithms for every problem in NP, which we take as strong evidence that such an algorithm is unlikely. We prove NP-hardness of P by giving a *polynomial-time reduction* from a known NP-hard problem Q ; this

reduction shows that a polynomial-time algorithm for P would give a polynomial-time algorithm for Q and hence for every problem in NP.

Most of the problems studied in database theory or in CSP research are obviously NP-hard, as they contain basic hard problems as special cases. Therefore, it is not completely obvious how to ask reasonable questions about NP-hardness where the answer is not trivial. One direction is to consider restricted parameter values. For example, for CSP problems, one may ask if the problem remains NP-hard if we restrict the domain size $|D|$ to 2 (yes, as 3SAT is still a special case), or restrict the constraints to binary (yes, 3-COLORING is a special case), or we apply both restrictions (no, with $|D| = 2$ and binary constraints the problem becomes the polynomial-time solvable 2SAT). Restricting the number $|V|$ of variables to any constant, say 10, gives a polynomial-time solvable special case even if the domain size $|D|$ is arbitrarily large (as we can try the at most $|D|^{10}$ possible assignments in polynomial time).

More generally, we can introduce restrictions on the problem in a systematic way and determine which of the restrictions lead to polynomial-time solvable and NP-hard special cases. In CSP research, a very well studied family of special cases arise from restricting the type of relations that are allowed in the constraints. Formally, let D be a finite domain and let $\mathcal{R}$ be a finite set of relations over D . Then we denote by $\text{CSP}(\mathcal{R})$ the special case of the general CSP problem where the instance is allowed to contain only constraints $c = \langle s, R \rangle$ where $R \in \mathcal{R}$. Equivalently, we can state this restriction in the language of the homomorphism problem in a very compact way. Let τ be a vocabulary and let $\mathbf{B}$ be a τ -structure. Then $\text{HOM}(_, \mathbf{B})$ is the special case of the general homomorphism problem where the input is a pair $(\mathbf{A}, \mathbf{B})$ of τ -structures, where $\mathbf{A}$ is arbitrary.

A classic result of Schaefer [59] characterized the complexity of $\text{CSP}(\mathcal{R})$ for any fixed finite set $\mathcal{R}$ of relation over the Boolean domain (i.e., $|D| = 2$). More precisely, Schaefer's Dichotomy Theorem showed that every such $\text{CSP}(\mathcal{R})$ problem is either polynomial-time solvable or NP-hard, and gave a clean characterization of the two cases. For several years, it was an outstanding open problem to prove an analog of this result for larger fixed domains (the Feder-Vardi Conjecture [35]). After partial progress, the conjecture was resolved in 2016 independently by Bulatov [21] and Zhuk [63].

While these classification results are cornerstones of modern CSP research, translating them into the language of database theory does not give much useful insight. Indeed, these characterization results would concern special cases where the domain of the attributes have constant size and every database relations is of constant size. From the viewpoint of database theory, a more relevant family of special cases can be obtained by restricting the structure of the query. Let us focus on **BOOLEAN JOIN QUERY**, the problem of deciding if the answer set is empty or not. If we assume, for example, that the primal graph of the query is a tree (acyclic graph), then it is easy to solve the problem in polynomial time, while the problem may remain NP-hard under other restrictions (for example, under the assumption that the primal graph has maximum degree 3 or is a planar graph etc.). Formally, in the language of CSPs, if $\mathcal{G}$ is any class of graphs, we may want to understand the complexity of the problem $\text{CSP}(\mathcal{G})$, which is CSP under the restriction that the primal graph belongs to class $\mathcal{G}$.

Can we identify every class $\mathcal{G}$ that leads to polynomial-time solvable special cases and use NP-hardness to give evidence of hardness for every other case? It is known that if $\mathcal{G}$ contains only graphs of bounded treewidth, then $\text{CSP}(\mathcal{G})$ becomes polynomial-time solvable. Treewidth is a combinatorial measure of graphs that can be thought of as a number expressing how tree-like the graph is: treewidth is 1 if and only if the graph is an acyclic forest, while other fixed values mean that the graph is similar to a tree with each node being replaced by a small graph. While the formal definition of treewidth is technical, it models very faithfully the requirements that make the algorithmic paradigm “split on small separators and recurse” work and its mathematical naturality is further evidenced by the fact that it was independently discovered in equivalent formulations at least three times [14, 41, 57]. The precise definition of treewidth is not essential for this paper; we include the definition here only for completeness.

Definition 4.1. A *tree decomposition* of a graph G is a pair $(\mathcal{B}, T)$ where T is a tree and $\mathcal{B} = \{B_t \mid t \in V(T)\}$ is a collection of subsets of $V(G)$ such that:

- $\bigcup_{t \in V(T)} B_t = V(G)$,
- for each edge $xy \in E(G)$, $\{x, y\} \subseteq B_t$ for some $t \in V(T)$;
- for each $x \in V(G)$ the set $\{t \mid x \in B_t\}$ induces a connected subtree of T .

The *width* of the tree decomposition is $\max_{t \in V(T)} \{|B_t| - 1\}$. The *treewidth* of a graph G is the minimum width over all tree decompositions of G . We denote by $\text{tw}(G)$ the treewidth of graph G .

Freuder [37] showed that if the primal graph has bounded treewidth, then the instance can be solved in polynomial time. By now, the result can be obtained by standard dynamic programming techniques on tree decompositions.

THEOREM 4.2 (FREUDER [37]). *For every fixed k , a CSP instance $I = (V, D, C)$ can be solved in time $O(|V| \cdot |D|^{k+1})$ if the primal graph has treewidth at most k .*

It follows from Theorem 4.2 that $\text{CSP}(\mathcal{G})$ is polynomial-time solvable if $\mathcal{G}$ has bounded treewidth, and it is easy to find graph classes $\mathcal{G}$ with unbounded treewidth (e.g., cliques) where the problem remains NP-hard. But, surprisingly, there seem to be cases that are neither polynomial-time solvable or NP-hard, thus a full classification into these two categories does not seem to be possible.

In the introduction, we mentioned that typically we expect that the problem at hand can be eventually classified as either polynomial-time solvable or NP-hard. While this may be true in most cases, there is no mathematical reason why this should be true in general. In fact, Ladner’s Theorem [48] states that if $P \neq NP$, then there are NP-intermediate problems in the class NP: problems that are neither polynomial-time solvable nor NP-hard. The proof of Ladner’s Theorem produces NP-intermediate problems that are highly artificial, so it is a different question whether there are natural problems that are NP-intermediate. There are two problems that are often highlighted as natural candidates for being NP-intermediate: GRAPH ISOMORPHISM and INTEGER FACTORIZATION. These two problems are not expected to be polynomial-time solvable, and the fact that they can be solved much more efficiently than brute force [10, 11, 49] suggests that they are not NP-hard either.

One could say that the reason why GRAPH ISOMORPHISM and INTEGER FACTORIZATION are NP-intermediate is that the deep algebraic and number-theoretic structures underlying these problems make them occupy a special place in the complexity landscape of NP problems. However, it is important to point out that problems can be (probably) NP-intermediate for more pedestrian reasons: it is possible to scale down an NP-hard problem in a way that it no longer NP-hard, but still not sufficiently easy to be polynomial-time solvable. We will refer to the following (artificial) example also in later sections.

Definition 4.3. A graph G is *special* if it has exactly two connected components: a clique of size k for some integer $k \geq 1$ and a path of exactly 2^k vertices. SPECIAL CSP and SPECIAL BOOLEAN JOIN QUERY are the restricted cases of the general problems where we assume that the primal graph is special.

Let us give some intuitive arguments why these problems could be NP-intermediate (we will make this more formal in later sections). First, the path part can be solved efficiently in polynomial time. Then we need to solve the clique part, which can certainly be done by brute force in time $O(n^k)$, where n is the total length of the input. But as already the primal graph has size larger than 2^k , we have $n \geq 2^k$ and hence $k \leq \log n$. Thus we can solve the problem in quasipolynomial time $n^{O(\log n)}$, which would be an exceptionally unusual property of an NP-hard problem. Moreover, it is not clear what substantial improvements we can expect on this algorithm: one would need to solve the clique part significantly faster than brute force. Therefore, it seems that these problems variants are likely to be NP-intermediate with best possible running time around $n^{O(\log n)}$.

This example shows that even if we just want to understand which special cases are polynomial-time solvable, then NP-hardness may not be sufficient for this purpose. As we shall see in later sections, we need to use other lower bound techniques for this type of classification. Additionally, these lower bound techniques can provide stronger lower bounds beyond just ruling out polynomial-time algorithms, showing the optimality of certain algorithms in a tighter way.

5 PARAMETERIZED INTRACTABILITY

Parameterized complexity considers algorithmic problems where each input instance has a parameter k associated with it. This parameter is typically either the size of the solution we are looking for or some measure of the input, such as the number of variables in a formula, the dimension of the input point set, the maximum degree of the input graph, or perhaps the alphabet size of the input strings. The central goal of parameterized complexity is to develop algorithms that are efficient on instances where the value of the parameter is small. Formally, we say that a parameterized problem is *fixed-parameter tractable* (FPT) if it can be solved in time $f(k) \cdot n^{O(1)}$, where n is the size of the input and f is a computable function depending only on k . This form of running time has to be contrasted with the running time $n^{O(k)}$ of brute force algorithms that are often easily achievable if k is the size of the solution we are looking for. Research in the past three decades has shown that

many of the natural NP-hard problems are FPT with various parameterizations, leading to algorithms that are often highly nontrivial and combinatorially deep [29, 31, 32, 36].

As an example, let us consider VERTEX COVER: given a graph G and an integer k , the task is to find a vertex cover S of size at most k , that is, a set S of at most k vertices such that every edge of G has at least one endpoint in S . Clearly, we can solve the problem by brute force on an n -vertex graph by trying each of the $O(n^k)$ sets S of size at most k . But more efficient algorithms are available: a standard application of the bounded-depth search tree technique already delivers a $2^k \cdot n^{O(1)}$ algorithm, which can be further refined with additional techniques [23]. This means that VERTEX COVER is FPT parameterized by the size of the solution. For CLIQUE, the trivial $O(n^k)$ brute force search can be improved to about $O(n^{\omega k/3})$ (where $\omega < 2.3729$ is the exponent for matrix multiplication [6, 53, 53]), but no FPT algorithm is known despite significant efforts.

Motivated by this apparent difference between VERTEX COVER and CLIQUE, Downey and Fellows introduced the notion of $W[1]$ -hardness and the $FPT \neq W[1]$ hypothesis [31]. We omit here the technical definitions related to the class $W[1]$; for the purpose of proving negative evidence, it is sufficient to know that $FPT \neq W[1]$ is equivalent to the statement “CLIQUE is not FPT” (or to “INDEPENDENT SET is not FPT”, as the two problems are equivalent by taking the complement of the graph). To define $W[1]$ -hardness, we need first the following notion of reduction:

Definition 5.1. Let P and Q be two parameterized problems. A *parameterized reduction* transforms an instance x of P with parameter k to an instance x' of Q with parameter k' such that

- (1) (x, k) is a yes-instance of P if and only if (x', k') is a yes-instance of Q .
- (2) The running time of the reduction is $f(k)|x|^{O(1)}$ for some computable function f .
- (3) We have $k' \leq f(k)$ for some computable function f .

The third requirement is what makes this notion very different from usual polynomial-time reductions: we have to pay extra attention not to blow up too much the parameter k in the reduction. Parameterized reductions were designed in a way that they transfer the property of being FPT: it can be shown that if there is a parameterized reduction from P to Q and Q is FPT, then P is FPT as well. We can define $W[1]$ -hardness by saying that a problem P is $W[1]$ -hard if there is a parameterized reduction from CLIQUE to P . We can interpret this as evidence that P is not FPT: an FPT algorithm for P would show that CLIQUE is FPT, violating the $FPT \neq W[1]$ hypothesis.

Let us have a look at the complexity of CSP via the lens of parameterized complexity. Given an instance $I = (V, D, C)$, we can introduce the number $k = |V|$ of variables as the parameter of the instance. We can decide if there is a solution by trying each of the $|D|^k = O(n^k)$ possible assignments. The NP-hardness of the problem implies that this cannot be improved to $n^{O(1)}$ (assuming $P \neq NP$), but this does not rule out the possibility that the problem is FPT parameterized by k , that is, there is a $f(k) \cdot n^{O(1)}$ time algorithm. Such an algorithm would be certainly of interest in contexts where we can assume that k is small, but D is large (which is typically

true in database applications). However, it is easy to see that the problem of finding a clique of size k in a graph G can be expressed as a CSP problem with k variables, $\binom{k}{2}$ constraints, and domain $D = V(G)$. That is, there is a parameterized reduction from CLIQUE to CSP parameterized by the number of variables, showing that the latter problem is unlikely to be FPT either.

We can now return to the question left open at the end of Section 4: what are those classes $\mathcal{G}$ for which $CSP(\mathcal{G})$ is polynomial-time solvable? As we have seen, NP-hardness does not seem to be sufficiently strong to highlight all the negative cases. Instead, let us look at the fixed-parameter tractability of $CSP(\mathcal{G})$, parameterized by the number k of variables. Now if $CSP(\mathcal{G})$ can be proved to be $W[1]$ -hard for some $\mathcal{G}$, then this implies in particular that it is not polynomial-time solvable, assuming $FPT \neq W[1]$.

For example, let $\mathcal{G}$ contain every special graph, as defined in Definition 4.3, and let us consider SPECIAL CSP (that is, $CSP(\mathcal{G})$) parameterized by the number of variables. Given an instance of CLIQUE (a graph G where it has to be decided if there is a clique of size k), then we can express it as a SPECIAL CSP instance as follows. We introduce k variables connected by $\binom{k}{2}$ binary constraints to express the problem of finding a k -clique, and additionally we introduce 2^k dummy variables connected by constraints forming a path. The primal graph is a k -clique plus a path on 2^k vertices, as required in SPECIAL CSP. The reduction turns the problem of finding a k -clique to a SPECIAL CSP instance on $f(k) = k + 2^k$ variables, hence this is a proper parameterized reduction. It follows that SPECIAL CSP is $W[1]$ -hard parameterized by the number of variables, and hence unlikely to be polynomial-time solvable.

More generally, Grohe, Schwentick, and Segoufin [40] proved that if $\mathcal{G}$ has unbounded treewidth, then $CSP(\mathcal{G})$ is $W[1]$ -hard, leading to a complete classification.

THEOREM 5.2 (GROHE, SCHWENTICK, AND SEGOUFIN [40]). *Let $\mathcal{G}$ be a decidable class of graphs. Assuming $FPT \neq W[1]$, the following are equivalent:*

- (1) $\mathcal{G}$ has bounded treewidth,
- (2) $CSP(\mathcal{G})$ is polynomial-time solvable,
- (3) $CSP(\mathcal{G})$ is FPT parameterized by the number k of variables.

Observe that there is a major coincidence here: the polynomial-time solvable cases are exactly the same as the FPT cases (which in principle could have been a more general class). This coincidence makes it possible to use $W[1]$ -hardness to identify those cases that are not polynomial-time solvable.

A more general result gives a classification in the framework of homomorphism problem for relational structures (which can be directly translated to results for BOOLEAN JOIN QUERY). Let τ be a vocabulary and let $\mathcal{A}$ be a class of τ -structures. Then $HOM(\mathcal{A}, _)$ is the special case of the general homomorphism problem where given two τ -structures (A, B) with $A \in \mathcal{A}$ and B arbitrary, the task is to decide if there is a homomorphism from A to B . The polynomial-time solvable cases again depend on treewidth, but in a slightly more complicated manner. If A' is a substructure of A such that there is a homomorphism from A to A' , then the problem instances (A, B) and (A', B) are equivalent. The smallest such substructure A' of A is called the *core* of A (it is known to be unique up to

isomorphism). It is the treewidth of this core that determines the complexity of the problem.

THEOREM 5.3 (GROHE [39]). *Let τ be a finite vocabulary and let $\mathcal{A}$ be a decidable class of τ -structures. Assuming $\text{FPT} \neq \text{W}[1]$, the following are equivalent:*

- (1) *the cores of the structures in $\mathcal{A}$ have bounded treewidth,*
- (2) *$\text{HOM}(\mathcal{A}, _)$ is polynomial-time solvable,*
- (3) *$\text{HOM}(\mathcal{A}, _)$ is FPT parameterized by the size k of the universe of $\mathcal{A}$.*

Again, we have that the polynomial-time solvable and FPT cases coincide, allowing the use of $\text{W}[1]$ -hardness for the classification of both properties.

6 THE EXPONENTIAL-TIME HYPOTHESIS

Parameterized complexity gives a finer understanding of the complexity of problems: for example, the negative results not only tell us that CLIQUE is not polynomial-time solvable, but they rule out algorithms with running time $f(k)n^{O(1)}$. However, this is still a qualitative result that rules out a certain running time, but does not tell us the exact complexity of the problem: can the n^k brute force search be improved to, say, $n^{\sqrt{k}}$, or to $n^{O(\log k)}$, or to $n^{O(\log \log \log k)}$, or to ...? For all we know, such algorithms cannot be ruled out based on the $\text{P} \neq \text{NP}$ or $\text{FPT} \neq \text{W}[1]$ conjectures. Similarly, for FPT problems such as VERTEX COVER where the best known algorithms have running time of the form $2^{O(k)} \cdot n^{O(1)}$, we cannot rule out that these algorithms can be significantly improved to, say, $2^{O(\sqrt{k})} \cdot n^{O(1)}$.

The Exponential-Time Hypothesis (ETH), formulated by Impagliazzo, Paturi, and Zane [43, 44], makes the assumption $\text{P} \neq \text{NP}$ more quantitative: informally, it not only tells us that NP-hard problems do not have polynomial-time algorithms, but it postulates that NP-hard problems really require exponential time and cannot be solved in subexponential time. The formal statement of the ETH is somewhat technical and for most applications it is more convenient to use the following assumption instead, which is an easy consequence of the ETH:

HYPOTHESIS 1 (CONSEQUENCE OF THE ETH, IMPAGLIAZZO, PATURI, AND ZANE [43, 44]). *3SAT with n variables cannot be solved in time $2^{o(n)}$.*

3SAT is the fundamental satisfiability problem where, given a Boolean formula in conjunctive normal form with at most 3 literals in each clause (e.g., $(x_1 \vee \bar{x}_3 \vee x_5) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3) \wedge (\bar{x}_2 \vee x_3 \vee x_4)$), the task is to decide whether a satisfying assignment exists. For completeness, let us recall the formal statement of the ETH, of which Hypothesis 1 is an easy consequence. Let s_k be the infimum of all real numbers δ for which there exists an $O(2^{\delta n})$ time algorithm for k -SAT. Then the ETH is the assumption that $s_k > 0$ for every $k \geq 3$. It is easy to show that this assumption implies Hypothesis 1, hence if we can show that some statement would refute Hypothesis 1, then it would refute the ETH as well.

As 3SAT can be interpreted as a special case of CSP with domain size 2 and constraints of arity 3, we can translate Hypothesis 1 into the language of CSPs to obtain a lower bound for solving instances with constant domain size.

COROLLARY 6.1. *Assuming the ETH, there is no algorithm that solves every CSP instance $I = (V, D, C)$ in time $2^{o(|V|)} \cdot n^{O(1)}$, even if $|D| = 2$ and every constraint has arity at most 3.*

Hypothesis 1 rules out the existence of algorithms that are subexponential in the number n of variables. But the number m of clauses in a 3SAT instance can be up to cubic in the number of variables, thus the length of the instance can be much larger than $O(n)$. Therefore, Hypothesis 1 does not rule out the existence of algorithms that are subexponential in the length of the instance: it could be potentially the case that all the really hard instances of 3SAT have, say, $\Omega(n^2)$ clauses, hence a $2^{o(\sqrt{n+m})}$ algorithm would be still compatible with Hypothesis 1. Impagliazzo, Paturi and Zane [44] showed that this is not the case: the Sparsification Lemma implies that, for the purposes of Hypothesis 1, 3SAT remains hard already when restricted to instances with a linear number of clauses. With the Sparsification Lemma, the following stronger assumption follows from Hypothesis 1:

HYPOTHESIS 2 (CONSEQUENCE OF THE ETH + SPARSIFICATION LEMMA, IMPAGLIAZZO, PATURI, AND ZANE [44]). *3SAT with n variables and m clauses cannot be solved in time $2^{o(n+m)}$.*

This stronger assumption turns out to be very useful to prove lower bounds for other problems. Reductions from 3SAT to other problems typically create instances whose size depends not only on the number n of variables, but also on the number m of clauses, hence it is important to have lower bounds on 3SAT in terms of both n and m . For example, if we look at textbook reductions from 3SAT to 3-COLORING, then they transform a formula with n variables and m clauses into a graph with $O(n+m)$ vertices and $O(n+m)$ edges. Such a reduction together with Hypothesis 2 implies a lower bound for binary CSP over a constant domain size.

COROLLARY 6.2. *Assuming the ETH, there is no algorithm that solves every CSP instance $I = (V, D, C)$ in time $2^{o(|V|+|C|)} \cdot n^{O(1)}$, even if $|D| = 3$ and every constraint is binary.*

Let us turn our attention now to parameterized problems. A key result in parameterized complexity states that, assuming ETH, the n^k brute force search for CLIQUE cannot be improved better than a constant factor in the exponent, even if we allow an arbitrary $f(k)$ factor in the running time.

THEOREM 6.3 (CHEN ET AL. [22]). *Assuming ETH, CLIQUE cannot be solved in time $f(k) \cdot n^{o(k)}$ for any computable function f .*

The same is true for the $\text{PARTITIONED CLIQUE}$, which, as we have seen in Section 2.3, is essentially equivalent to a binary CSP instance where the primal graph is clique. Therefore, we can translate Theorem 6.3 into the language of CSPs.

THEOREM 6.4. *Assuming ETH, there is no algorithm that solves every binary CSP instance $I = (V, D, C)$ in time $f(|V|) \cdot |D|^{o(|V|)} \cdot n^{O(1)}$, where f is an arbitrary computable function.*

Moreover, we have seen in Section 5 that k - CLIQUE can be reduced to a SPECIAL CSP instance with $k + 2^k$ variables. Together with Theorem 6.3 it follows that, assuming the ETH, there is no $f(|V|)n^{o(\log |V|)}$ time algorithm for SPECIAL CSP . This makes the NP-intermediate status of the problem very precise: it is indeed

$n^{O(\log |V|)}$ the best possible running time we can hope for this problem.

Note that the treewidth of a k -clique is $k - 1$. Therefore, Theorem 6.4 shows that if k is the treewidth of the primal graph of the CSP instance, then the $n^{O(k)}$ time algorithm of Freuder [37] is essentially optimal in the sense that the exponent cannot be improved by more than a constant factor.

THEOREM 6.5. *Assuming ETH, there is no algorithm that solves every binary CSP instance $I = (V, D, C)$ in time $f(|V|) \cdot n^{o(k)}$, where k is the treewidth of the primal graph and f is an arbitrary computable function f .*

One could interpret Theorem 6.5 as saying that the treewidth-based algorithm of Freuder is an optimal way of solving CSP instances. However, this interpretation is misleading. What Theorem 6.5 really says is that there is one type of primal graphs, namely cliques, where the $n^{O(k)}$ running time that follows using that treewidth-based algorithm is essentially optimal. This *does not* rule out the possibility that there are some graph classes, maybe planar graphs, bounded-degree graphs, interval graphs, etc. where it is possible to solve the problem $n^{o(k)}$ time, where k is the treewidth of the primal graph. Formally, we can approach this possibility in the spirit of Theorem 5.2, by considering the problem $\text{CSP}(\mathcal{G})$, where the primal graph is restricted some class $\mathcal{G}$. The following lower bound shows that the treewidth-based algorithm is still optimal for any such $\text{CSP}(\mathcal{G})$, up to a logarithmic factor in the exponent.

THEOREM 6.6 ([52]). *Let $\mathcal{G}$ be a class of graphs with unbounded treewidth. Assuming ETH, there is no algorithm that solves every instance $I = (V, D, C)$ of $\text{CSP}(\mathcal{G})$ in time $f(|V|)n^{o(k/\log k)}$, where k is the treewidth of the primal graph and f is an arbitrary computable function.*

The formulation of Theorem 6.6 was originally chosen in a way to be analogous to the formulation of Theorem 5.2. However, it later turned out that it is possible to state it in a slightly more robust and expressive way that shows the precise complexity of individual primal graphs.

THEOREM 6.7 ([25]). *Assuming the ETH, there exists a universal constant $\alpha > 0$ such that for any fixed primal graph G with treewidth $k \geq 2$, there is no algorithm deciding the binary CSP instances $I = (V, D, C)$ whose primal graph is G in time $O(|D|^{\alpha \cdot k / \log k})$.*

7 THE STRONG EXPONENTIAL-TIME HYPOTHESIS

Despite the usefulness of the ETH, there are complexity lower bounds that seem to be beyond the reach of what can be proved as a consequence of this hypothesis. Impagliazzo, Paturi, and Zane [44] proposed an even stronger assumption on the complexity of NP-hard problems: the so-called Strong Exponential-Time Hypothesis (SETH). Using the notation introduced at the beginning of Section 6, the SETH assumes that $\lim_{k \rightarrow \infty} s_k = 1$. The following consequence of the SETH is a convenient formulation that can be used as a starting point for lower bounds on other problems:

HYPOTHESIS 3 (CONSEQUENCE OF THE SETH, IMPAGLIAZZO, PATURI, AND ZANE [44]). *SAT with n variables and m clauses cannot be solved in time $(2 - \epsilon)^n \cdot m^{O(1)}$ for any $\epsilon > 0$.*

Intuitively, Hypothesis 3 states that there is no better algorithm for SAT than the brute force search of trying each of the 2^n possible assignments. Note that here SAT is the satisfiability problem with unbounded clause length. For fixed clause length, algorithms better than 2^n are known: for example, the best known algorithms for 3SAT and 4SAT have running times 1.308^n and 1.469^n , respectively [42]. The SETH states that the base of the exponent has to get closer and closer to 1 as the clause length increases, and it is not possible to have an algorithm with base $2 - \epsilon$ that works for arbitrary large clause length.

It is important to note that there is no known analogue of the Sparsification Lemma for the SETH. That is, we cannot assume that the hard instances stipulated by Hypothesis 3 have only a linear number of clauses: for all we know, the number of clauses can be exponential in the number n of variables. This severely limits the applicability of lower bounds based on the SETH as any reduction from the SAT instance would create instances whose sizes are potentially exponentially large in n . Nevertheless, the SETH has found applications in parameterized complexity, for example, giving tight lower bounds on how the running time has to depend on treewidth [15, 27, 28, 30, 33, 45, 46, 51].

An important parameterized problem for which the SETH gives a very tight lower bound is DOMINATING SET. The *closed neighborhood* $N[v] = N(v) \cup \{v\}$ of a vertex v consists of the vertex itself and its neighbors. A *dominating set* S is a set of vertices that contains a vertex from the closed neighborhood of every vertex, in other words, every vertex is either selected or has a selected neighbor. In the DOMINATING SET problem, given a graph G and an integer k , the task is to find a dominating set S of size at most k . If G is an n -vertex graph, then the trivial brute force algorithm enumerates the $O(n^k)$ subsets of size at most k and needs $O(n^2)$ time for each of them to check if they form a solution. This results in a $O(n^{k+2})$ time algorithm, which can be improved to $n^{k+o(1)}$ [34]. The following lower bound shows that any small constant improvement beyond k in the exponent would violate the SETH.

THEOREM 7.1 (PATRASCU AND WILLIAMS [56]). *If there is an integer $k \geq 3$ and a real number $\epsilon > 0$ such that k -DOMINATING SET can be solved in time $O(n^{k-\epsilon})$ on n -vertex graphs, then the SETH is false.*

As a demonstration, we show how Theorem 7.1 allows us to make Theorem 6.5 tighter. We are not just ruling out $|D|^{o(k)}$ time, but any potential improvement in the exponent of the domain size beyond k , getting closer to the upper bound of Theorem 4.2.

THEOREM 7.2. *If there are integers $k \geq 3$, $c \geq 1$, and a real number $\epsilon > 0$ such that there is an algorithm solving CSP instances $I = (V, D, C)$ whose primal graph has treewidth at most k in time $O(|V|^c \cdot |D|^{k-\epsilon})$, then the SETH is false.*

PROOF. Let $g \geq 1$ be the smallest integer such that $ge > c + \epsilon$ and let $t = gk$. First we present a generic reduction from t -DOMINATING SET on an n -vertex graph to a CSP instance with domain size n where the treewidth of the primal graph is t . Let G be an n -vertex graph where we need to find a solution S of size at most t . For simplicity of notation, let us assume that $V(G) = [n]$. We construct a CSP instance $I = (V, D, C)$ the following way. The set V contains $t + n$ variables $s_1, \dots, s_t, x_1, \dots, x_n$. The domain D is $V(G) = [n]$. The intended meaning of the value of s_i is the i -th vertex of the

solution, and the intended meaning of $x_j = i$ is that the solution vertex represented by s_i is in $N[j]$. To enforce this interpretation, for every $i \in [t]$ and $j \in [n]$, we introduce a constraint $c_{i,j} = \langle (s_i, v_j), R_{i,j} \rangle$, where

$$R_{i,j} = \{(a, b) \mid a \in [n], b \in [t], b \neq i\} \cup \{(a, b) \mid a \in [n], b \in [t], b = i, a \in N[j]\}$$

It is not difficult to check that if there is a solution to this CSP instance, then $\{s_1, \dots, s_t\}$ is a dominating set (as vertex s_{x_j} is in the closed neighborhood $N[j]$ of j). Conversely, a solution S of t -DOMINATING SET can be turned into a solution of this CSP instance. Observe that the primal graph is complete bipartite graph with t vertices on one side and n vertices on the other side. Such a graph has treewidth at most t .

To obtain the required form of the lower bound, we need to modify the constructed CSP instance. Let us group the variables $x_1, \dots, x_t$ into $t/g = k$ groups of size g each. If we increase the domain from D to D^g (having size n^g), we can represent each group with a single new variable (and modify the constraints accordingly). This way, we can obtain an equivalent CSP instance $I' = (V', D', C')$ where $|D'| = n^g$ and treewidth of the primal graph is at most k . By our assumption, this CSP instance I' (and hence the original t -DOMINATING SET instance) can be solved in time

$$\begin{aligned} O(|V'|^c \cdot |D'|^{k-\epsilon}) &= O(n^c \cdot n^{g(t/g-\epsilon)}) \\ &= O(n^{t+c-g\epsilon}) = O(n^{t-\epsilon}). \end{aligned}$$

The size of the constructed instance I' can be generously bounded by $O(n^{2g+1})$ which is less than $O(n^{t-\epsilon})$. The reduction presented above can be done in time linear in the size of I' , hence the running time of the reduction itself is dominated by the running time of solving I' with the assumed algorithm. Therefore, we obtain an algorithm for solving t -DOMINATING SET in time $O(n^{t-\epsilon})$. By Theorem 7.1, this violates SETH. $\square$

In recent years, the SETH has been successfully used to give lower bounds for polynomial-time solvable problems, for example, by showing that the textbook $O(n^2)$ dynamic programming algorithm for EDIT DISTANCE cannot be significantly improved: it cannot be solved in time $O(n^{2-\epsilon})$ for any $\epsilon > 0$, unless the SETH fails [12, 19]. Many other tight results of this form can be found in the recent literature under the name “fine-grained complexity” [1, 3, 5, 12, 17–20, 56, 58, 62].

8 OTHER CONJECTURES

We finish the overview of lower bound techniques with a few other complexity conjectures that have appeared recently in the literature. This section does not contain any strong results or nontrivial reductions; the goal is to present assumptions that have direct consequences when translated into the language of database theory and CSP. The aim is to raise awareness of the existence of these conjectures, which may be the starting point of future research.

The k -clique conjecture. Matrix multiplication techniques can be used to detect if a graph contains a triangle: if A is the adjacency matrix of G , then G contains a triangle if and only if A^3 has a nonzero value on the diagonal. Therefore, if we have an algorithm

for multiplying two $n \times n$ matrices in time $O(n^\omega)$ for some ω (the current best known algorithm has $\omega < 2.3729$ [6]), then we can detect in time $O(n^3)$ if an n -vertex graph contains a triangle. Nešetřil and Poljak [53] showed in 1985 that this can be generalized further for detecting a clique of size k in time $O(n^{\omega k/3})$ (if k is divisible by 3). As no significant improvement over this approach appeared in the past 35 years, one can conjecture that there is no $O(n^{(\omega-\epsilon)k/3+c})$ time algorithm for k -CLIQUE for any $\epsilon, c > 0$. Abboud, Backurs, and Vassilevska Williams [2] used this conjecture to give evidence that Valiant’s $O(n^\omega)$ time parsing algorithm [60] from 1975 is optimal.

As discussed in Section 2.3, k -CLIQUE on an n -vertex graph can be represented as a CSP with k variables, $\binom{k}{2}$ constraints, and domain size n . Therefore, the k -clique conjecture further refines Theorem 6.4 by ruling out not only a $|D|^{o(|V|)}$ dependence in the running time, but also $|D|^{(\omega-\epsilon)|V|/3+c}$ for any $\epsilon, c > 0$.

The d -uniform hyperclique conjecture. A hypergraph is d -uniform if every hyperedge contains exactly d vertices. The analog of a k -clique in a d -uniform hypergraph is a set S of k vertices such that each of the $\binom{k}{d}$ possible hyperedges are present in S . Somewhat surprisingly, matrix multiplication techniques seem to speed up the search for k -cliques only for $d = 2$ (ordinary graphs). For any fixed $d \geq 3$, nothing substantially better is known than trying every set of size k . This suggests the conjecture that there is no $O(n^{(1-\epsilon)k+c})$ time algorithm for detecting k -cliques in d -uniform hypergraphs for any fixed $d \geq 2$ and $\epsilon, c > 0$ [50]. We can again translate this conjecture into the language of CSPs: we can show that even if the arity of every constraint is at most 3, there is no $f(|V|) \cdot |D|^{(1-\epsilon)|V|+c} \cdot n^{O(1)}$ algorithm for CSP for any $\epsilon, c > 0$ and computable function f . Therefore, we get very tight lower bounds showing that essentially the brute force search of all assignments cannot be avoided. The d -uniform hyperclique conjecture was used to rule out the possibility of constant-delay enumeration algorithms [13, 16].

The triangle conjecture. In database query problems it is more relevant to express the running time in terms of the size of the database relations rather than the size of the domain. For example, given the query $Q = R_1(a_1, a_2) \bowtie R_2(a_1, a_3) \bowtie R_3(a_2, a_3)$ whose primal graph is the triangle, matrix multiplication can be used to check in time $O(d^\omega)$ if the answer is empty, where d is the size of the domain of the attributes. But what can we say about the running time expressed as a function N of the maximum size of the relations? As we have seen in Section 3, the size of the solution is $O(N^{3/2})$, we can enumerate it in time $O(N^{3/2})$, and this is tight. However, this does not rule out the possibility that there are faster algorithms for deciding if the answer is empty. In particular, can this be solved in linear time? Note that this question is equivalent to asking for the best possible running time for detecting triangles, where the running time is now expressed as a function of the number m of edges. The best known algorithm of this form detects the existence of a triangle in time $O(m^{2\omega/(w+1)})$ [7] and one can state as a conjecture (Strong Triangle Conjecture [4]) that this is indeed best possible.

9 CONCLUSIONS

We have seen a sequence of complexity lower bounds of various strengths. The results were based on assumptions with different levels of plausibility, going all the way from unconditional bounds, classic NP-completeness, to novel conjectures. The results also differ in the tightness of the lower bound: they can be only qualitative results (polynomial-time vs. NP-hard, FPT vs. $W[1]$ -hard) or quantitative lower bounds showing the optimality of current algorithms to various levels of tightness.

What can we learn from all these results? First, when aiming for a lower bound, we need to select a precise form of the bound. Ideally, we would like to have negative results that rule out the possibility of any improved algorithm compared to what is known currently, showing that they are already optimal. The meaning of “any improved algorithm” needs to be clarified precisely and the choice of this meaning can greatly influence the technical difficulty of the lower bound proof and the required assumptions. Second, we need to choose a suitable complexity assumption that we can base the result on. There are established conjectures, such as the ETH, that are widely used in different domains. But we should be ready to connect our database theory or CSP problem at hand with other, less celebrated open question as well. The general theme of conditional lower bounds is to transform a relatively specialized question to a more fundamental question that was studied from multiple directions. If we can formally establish that the main challenge in understanding our problem is really some other, more fundamental problem in disguise, then this means that spending further efforts on finding improved algorithms is not timely and we can assume for the time being that any algorithm matching the lower bound is optimal. This is a common situation in complexity theory: as it is often said, computational complexity progresses by reducing the number of questions, without increasing the number of answers.

ACKNOWLEDGMENTS

Research supported by the European Research Council (ERC) consolidator grant No. 725978 SYSTEMATICGRAPH.

REFERENCES

- [1] Amir Abboud, Arturs Backurs, and Virginia Vassilevska Williams. 2015. Tight Hardness Results for LCS and Other Sequence Similarity Measures. In *Proceedings of the 56th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2015)*. 59–78.
- [2] Amir Abboud, Arturs Backurs, and Virginia Vassilevska Williams. 2018. If the Current Clique Algorithms Are Optimal, so Is Valiant’s Parser. *SIAM J. Comput.* 47, 6 (2018), 2527–2555. <https://doi.org/10.1137/16M1061771>
- [3] Amir Abboud, Karl Bringmann, Holger Dell, and Jesper Nederlof. 2018. More consequences of falsifying SETH and the orthogonal vectors conjecture. In *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2018)*, Los Angeles, CA, USA, June 25–29, 2018, Ilias Diakonikolas, David Kempe, and Monika Henzinger (Eds.). ACM, 253–266. <https://doi.org/10.1145/3188745.3188938>
- [4] Amir Abboud and Virginia Vassilevska Williams. 2014. Popular Conjectures Imply Strong Lower Bounds for Dynamic Problems. In *55th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2014, Philadelphia, PA, USA, October 18–21, 2014*. IEEE Computer Society, 434–443. <https://doi.org/10.1109/FOCS.2014.53>
- [5] Amir Abboud, Virginia Vassilevska Williams, and Huacheng Yu. 2018. Matching Triangles and Basing Hardness on an Extremely Popular Conjecture. *SIAM J. Comput.* 47, 3 (2018), 1098–1122. <https://doi.org/10.1137/15M1050987>
- [6] Josh Alman and Virginia Vassilevska Williams. 2021. A Refined Laser Method and Faster Matrix Multiplication. In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms, SODA 2021, Virtual Conference, January 10 - 13, 2021*, Daniel Marx (Ed.). SIAM, 522–539. <https://doi.org/10.1137/1.9781611976465.32>
- [7] Noga Alon, Raphael Yuster, and Uri Zwick. 1997. Finding and Counting Given Length Cycles. *Algorithmica* 17, 3 (1997), 209–223. <https://doi.org/10.1007/BF02523189>
- [8] Sanjeev Arora and Boaz Barak. 2009. *Computational Complexity - A Modern Approach*. Cambridge University Press. <http://www.cambridge.org/catalogue/catalogue.asp?isbn=9780521424264>
- [9] Albert Atserias, Martin Grohe, and Daniel Marx. 2013. Size Bounds and Query Plans for Relational Joins. *SIAM J. Comput.* 42, 4 (2013), 1737–1767. <https://doi.org/10.1137/110859440>
- [10] László Babai. 2016. Graph isomorphism in quasipolynomial time [extended abstract]. In *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2016, Cambridge, MA, USA, June 18–21, 2016*, Daniel Wichs and Yishay Mansour (Eds.). ACM, 684–697. <https://doi.org/10.1145/2897518.2897542>
- [11] László Babai and Eugene M. Luks. 1983. Canonical Labeling of Graphs. In *Proceedings of the 15th Annual ACM Symposium on Theory of Computing, 25–27 April, 1983, Boston, Massachusetts, USA*, David S. Johnson, Ronald Fagin, Michael L. Fredman, David Harel, Richard M. Karp, Nancy A. Lynch, Christos H. Papadimitriou, Ronald L. Rivest, Walter L. Ruzzo, and Joel I. Seiferas (Eds.). ACM, 171–183. <https://doi.org/10.1145/800061.808746>
- [12] Arturs Backurs and Piotr Indyk. 2018. Edit Distance Cannot Be Computed in Strongly Subquadratic Time (Unless SETH is False). *SIAM J. Comput.* 47, 3 (2018), 1087–1097. <https://doi.org/10.1137/15M1053128>
- [13] Guillaume Bagan, Arnaud Durand, and Etienne Grandjean. 2007. On Acyclic Conjunctive Queries and Constant Delay Enumeration. In *Computer Science Logic, 21st International Workshop, CSL 2007, 16th Annual Conference of the EACSL, Lausanne, Switzerland, September 11–15, 2007, Proceedings (Lecture Notes in Computer Science)*, Jacques Duparc and Thomas A. Henzinger (Eds.), Vol. 4646. Springer, 208–222. https://doi.org/10.1007/978-3-540-74915-8_18
- [14] Umberto Bertele and Francesco Brioschi. 1973. On Non-serial Dynamic Programming. *J. Comb. Theory, Ser. A* 14, 2 (1973), 137–148. [https://doi.org/10.1016/0097-3165\(73\)90016-2](https://doi.org/10.1016/0097-3165(73)90016-2)
- [15] Glencora Borradaile and Hung Le. 2016. Optimal Dynamic Program for r-Domination Problems over Tree Decompositions. In *11th International Symposium on Parameterized and Exact Computation (IPEC 2016) (LIPIcs)*, Jiong Guo and Danny Hermelin (Eds.), Vol. 63. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 8:1–8:23. <https://doi.org/10.4230/LIPIcs.IPEC.2016.8>
- [16] Johann Brault-Baron. 2013. *De la pertinence de l’énumération : complexité en logiques propositionnelle et du premier ordre. (The relevance of the list: propositional logic and complexity of the first order)*. Ph.D. Dissertation. University of Caen Normandy, France. <https://tel.archives-ouvertes.fr/tel-01081392>
- [17] Karl Bringmann. 2014. Why Walking the Dog Takes Time: Frechet Distance Has No Strongly Subquadratic Algorithms Unless SETH Fails. In *55th IEEE Annual Symposium on Foundations of Computer Science (FOCS 2014)*, Philadelphia, PA, USA, October 18–21, 2014. IEEE Computer Society, 661–670. <https://doi.org/10.1109/FOCS.2014.76>
- [18] Karl Bringmann, Pawel Gawrychowski, Shay Mozes, and Oren Weimann. 2018. Tree Edit Distance Cannot be Computed in Strongly Subcubic Time (unless APSP can). In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2018)*, Artur Czumaj (Ed.). SIAM, 1190–1206. <https://doi.org/10.1137/1.9781611975031.77>
- [19] Karl Bringmann and Marvin Künnemann. 2015. Quadratic Conditional Lower Bounds for String Problems and Dynamic Time Warping. In *IEEE 56th Annual Symposium on Foundations of Computer Science (FOCS 2015)*, Berkeley, CA, USA, 17–20 October, 2015, Venkatesan Guruswami (Ed.). IEEE Computer Society, 79–97. <https://doi.org/10.1109/FOCS.2015.15>
- [20] Karl Bringmann and Marvin Künnemann. 2018. Multivariate Fine-Grained Complexity of Longest Common Subsequence. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2018)*, Artur Czumaj (Ed.). SIAM, 1216–1235. <https://doi.org/10.1137/1.9781611975031.79>
- [21] Andrei A. Bulatov. 2017. A Dichotomy Theorem for Nonuniform CSPs. In *58th IEEE Annual Symposium on Foundations of Computer Science (FOCS 2017)*, Berkeley, CA, USA, October 15–17, 2017, Chris Umans (Ed.). IEEE Computer Society, 319–330. <https://doi.org/10.1109/FOCS.2017.37>
- [22] Jianer Chen, Xiuzhen Huang, Iyad A. Kanj, and Ge Xia. 2004. Linear FPT reductions and computational lower bounds. In *Proceedings of the 36th Annual ACM Symposium on Theory of Computing (STOC 2004)*. ACM, New York, 212–221.
- [23] Jianer Chen, Iyad A. Kanj, and Weijia Jia. 2001. Vertex cover: further observations and further improvements. *J. Algorithms* 41, 2 (2001), 280–301.
- [24] F. R. K. Chung, R. L. Graham, P. Frankl, and J. B. Shearer. 1986. Some intersection theorems for ordered sets and graphs. *J. Combin. Theory Ser. A* 43, 1 (1986), 23–37.
- [25] Vincent Cohen-Addad, Éric Colin de Verdière, Daniel Marx, and Arnaud de Mesmay. [n.d.]. Almost Tight Lower Bounds for Hard Cutting Problems in Embedded Graphs. To appear in *Journal of the ACM*.
- [26] Stephen A. Cook. 1971. The Complexity of Theorem-Proving Procedures. In *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing, May 3–5, 1971, Shaker Heights, Ohio, USA*, Michael A. Harrison, Ranan B. Banerji, and Jeffrey D. Ullman (Eds.). ACM, 151–158. <https://doi.org/10.1145/800157.805047>

- [27] Radu Curticapean and Dániel Marx. 2016. Tight conditional lower bounds for counting perfect matchings on graphs of bounded treewidth, cliquewidth, and genus. In *Proceedings of the 27th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2016)*. 1650–1669.
- [28] Marek Cygan, Holger Dell, Daniel Lokshantov, Dániel Marx, Jesper Nederlof, Yoshio Okamoto, Ramamohan Paturi, Saket Saurabh, and Magnus Wahlström. 2016. On Problems as Hard as CNF-SAT. *ACM Trans. Algorithms* 12, 3 (2016), 41:1–41:24.
- [29] Marek Cygan, Fedor V. Fomin, Lukasz Kowalik, Daniel Lokshantov, Dániel Marx, Marcin Pilipczuk, Michal Pilipczuk, and Saket Saurabh. 2015. *Parameterized Algorithms*. Springer.
- [30] Marek Cygan, Jesper Nederlof, Marcin Pilipczuk, Michal Pilipczuk, Johan M. M. van Rooij, and Jakub Onufry Wojtaszczyk. 2011. Solving Connectivity Problems Parameterized by Treewidth in Single Exponential Time. In *Proceedings of the 52nd Annual IEEE Symposium on Foundations of Computer Science (FOCS 2011)*. 150–159.
- [31] Rodney G. Downey and Michael R. Fellows. 1999. *Parameterized Complexity*. Springer, New York. xvi+533 pages.
- [32] Rodney G. Downey and Michael R. Fellows. 2013. *Fundamentals of Parameterized Complexity*. Springer. <https://doi.org/10.1007/978-1-4471-5559-1>
- [33] László Egri, Dániel Marx, and Paweł Rzażewski. 2018. Finding List Homomorphisms from Bounded-treewidth Graphs to Reflexive Graphs: a Complete Complexity Characterization. In *35th Symposium on Theoretical Aspects of Computer Science (STACS 2018)*. 27:1–27:15. <https://doi.org/10.4230/LIPIcs.STACS.2018.27>
- [34] Friedrich Eisenbrand and Fabrizio Grandoni. 2004. On the complexity of fixed parameter clique and dominating set. *Theor. Comput. Sci.* 326, 1–3 (2004), 57–67. <https://doi.org/10.1016/j.tcs.2004.05.009>
- [35] Tomás Feder and Moshe Y. Vardi. 1999. The computational structure of monotone monadic SNP and constraint satisfaction: a study through Datalog and group theory. *SIAM J. Comput.* 28, 1 (1999), 57–104.
- [36] Jörg Flum and Martin Grohe. 2006. *Parameterized Complexity Theory*. Springer, Berlin. xiv+493 pages.
- [37] E. C. Freuder. 1990. Complexity of K-Tree Structured Constraint Satisfaction Problems. In *Proc. of AAAI-90*. Boston, MA, 4–9.
- [38] M. R. Garey and David S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- [39] Martin Grohe. 2007. The complexity of homomorphism and constraint satisfaction problems seen from the other side. *J. ACM* 54, 1 (2007), 1. <https://doi.org/10.1145/1206035.1206036>
- [40] Martin Grohe, Thomas Schwentick, and Luc Segoufin. 2001. When is the evaluation of conjunctive queries tractable?. In *Proceedings of the thirty-third annual ACM symposium on Theory of computing (STOC 2001)* (Hersonissos, Greece). ACM Press, New York, NY, USA, 657–666. <https://doi.org/10.1145/380752.380867>
- [41] Rudolf Halin. 1976. S-functions for graphs. *Journal of Geometry* 8, 1–2 (1976), 171–186.
- [42] Timon Hertli. 2014. 3-SAT Faster and Simpler — Unique-SAT Bounds for PPSZ Hold in General. *SIAM J. Comput.* 43, 2 (2014), 718–729. <https://doi.org/10.1137/120868177>
- [43] Russell Impagliazzo and Ramamohan Paturi. 2001. On the Complexity of k -SAT. *J. Comput. Syst. Sci.* 62, 2 (2001), 367–375.
- [44] Russell Impagliazzo, Ramamohan Paturi, and Francis Zane. 2001. Which problems have strongly exponential complexity? *J. Comput. System Sci.* 63, 4 (2001), 512–530.
- [45] Yoichi Iwata and Yuichi Yoshida. 2015. On the Equivalence among Problems of Bounded Width. In *23rd Annual European Symposium (ESA 2015) (Lecture Notes in Computer Science)*, Nikhil Bansal and Irene Finocchi (Eds.), Vol. 9294. Springer, 754–765. https://doi.org/10.1007/978-3-662-48350-3_63
- [46] Lars Jaffke and Bart M. P. Jansen. 2017. Fine-Grained Parameterized Complexity Analysis of Graph Coloring Problems. In *Proceedings of the 10th International Conference on Algorithms and Complexity (CIAC 2017) (Lecture Notes in Computer Science)*, Vol. 10236. 345–356.
- [47] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In *Proceedings of a symposium on the Complexity of Computer Computations, held March 20–22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, USA (The IBM Research Symposia Series)*, Raymond E. Miller and James W. Thatcher (Eds.). Plenum Press, New York, 85–103. https://doi.org/10.1007/978-1-4684-2001-2_9
- [48] Richard E. Ladner. 1975. On the Structure of Polynomial Time Reducibility. *J. ACM* 22, 1 (1975), 155–171. <https://doi.org/10.1145/321864.321877>
- [49] H. W. Lenstra, Jr. and Carl Pomerance. 1992. A rigorous time bound for factoring integers. *J. Amer. Math. Soc.* 5, 3 (1992), 483–516. <https://doi.org/10.2307/2152702>
- [50] Andrea Lincoln, Virginia Vassilevska Williams, and R. Ryan Williams. 2018. Tight Hardness for Shortest Cycles and Paths in Sparse Graphs. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2018, New Orleans, LA, USA, January 7–10, 2018*, Artur Czumaj (Ed.). SIAM, 1236–1252. <https://doi.org/10.1137/1.9781611975031.80>
- [51] Daniel Lokshantov, Dániel Marx, and Saket Saurabh. 2018. Known Algorithms on Graphs of Bounded Treewidth Are Probably Optimal. *ACM Trans. Algorithms* 14, 2 (2018), 13:1–13:30. <https://doi.org/10.1145/3170442>
- [52] Dániel Marx. 2010. Can You Beat Treewidth? *Theory of Computing* 6, 1 (2010), 85–112. <https://doi.org/10.4086/toc.2010.v006a005> arXiv:toc:v006/a005
- [53] Jaroslav Nešetřil and Svatopluk Poljak. 1985. On the complexity of the subgraph problem. *Commentationes Mathematicae Universitatis Carolinae* 26, 2 (1985), 415–419.
- [54] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2018. Worst-case Optimal Join Algorithms. *J. ACM* 65, 3 (2018), 16:1–16:40. <https://doi.org/10.1145/3180143>
- [55] C. H. Papadimitriou. 1994. *Computational Complexity*. Addison Wesley.
- [56] Mihai Patrascu and Ryan Williams. 2010. On the Possibility of Faster SAT Algorithms. In *Proceedings of the 21st Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2010)*. 1065–1075.
- [57] Neil Robertson and Paul D. Seymour. 1984. Graph minors. III. Planar tree-width. *J. Comb. Theory, Ser. B* 36, 1 (1984), 49–64. [https://doi.org/10.1016/0095-8956\(84\)90013-3](https://doi.org/10.1016/0095-8956(84)90013-3)
- [58] Liam Roditty and Virginia Vassilevska Williams. 2013. Fast approximation algorithms for the diameter and radius of sparse graphs. In *Proceedings of the 45th Annual ACM on Symposium on Theory of Computing (STOC 2013)*. 515–524.
- [59] Thomas J. Schaefer. 1978. The Complexity of Satisfiability Problems. In *Proceedings of the 10th Annual ACM Symposium on Theory of Computing, May 1–3, 1978, San Diego, California, USA*, Richard J. Lipton, Walter A. Burkhard, Walter J. Savitch, Emily P. Friedman, and Alfred V. Aho (Eds.). ACM, 216–226. <https://doi.org/10.1145/800133.804350>
- [60] Leslie G. Valiant. 1975. General Context-Free Recognition in Less than Cubic Time. *J. Comput. Syst. Sci.* 10, 2 (1975), 308–315. [https://doi.org/10.1016/S0022-0000\(75\)80046-8](https://doi.org/10.1016/S0022-0000(75)80046-8)
- [61] Todd L. Veldhuizen. 2014. Triejoin: A Simple, Worst-Case Optimal Join Algorithm. In *Proc. 17th International Conference on Database Theory (ICDT 2014)*. 96–106. <https://doi.org/10.5441/002/icdt.2014.13>
- [62] Virginia Vassilevska Williams. 2015. Hardness of Easy Problems: Basing Hardness on Popular Conjectures such as the Strong Exponential Time Hypothesis. In *Proceedings of the 10th International Symposium on Parameterized and Exact Computation (IPEC 2015) (LIPIcs)*, Vol. 43. 17–29.
- [63] Dmitriy Zhuk. 2017. A Proof of CSP Dichotomy Conjecture. In *58th IEEE Annual Symposium on Foundations of Computer Science (FOCS 2017)*, Berkeley, CA, USA, October 15–17, 2017. 331–342. <https://doi.org/10.1109/FOCS.2017.38>